Day 3 : Git , Github , Flutter Basis

Learning Objectives

- Master Git for Flutter project version control.
- Understand GitHub repository management.
- Learn Flutter architecture and widget tree.
- Differentiate between Stateless and Stateful widgets.
- Build basic Flutter user interfaces.
- Create and manage Flutter projects using Git and GitHub.

1. Git for Flutter Projects

Introduction

Git is a distributed version control system that helps developers track code changes, collaborate with teams, and maintain project history. In Flutter development, Git is used to manage source code efficiently and keep backups of project versions.

```
C:\Users\Sneha>flutter create demo
Recreating project demo...
Resolving dependencies in `demo`... (2.2s)
Downloading packages...
Got dependencies in `demo`.
Wrote 3 files.

All done!
You can find general documentation for Flutter at:
https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider:
https://www.youtube.com/c/flutterdev

In order to run your application, type:

  $ cd demo
  $ flutter run

Your application code is in demo\lib\main.dart.


C:\Users\Sneha>cd demo

C:\Users\Sneha\demo>git init
Initialized empty Git repository in C:/Users/Sneha/demo/.git/
```

1. **flutter create demo :** Creates a new Flutter project named **demo**
2. **cd demo :** Changes directory into the demo project folder

Git Command Start :

3. **git add .** → Stages all changes in the project for commit.
4. **git status** → Shows current status of files in the repository.
5. **git commit -m "Initial Flutter project setup"** → Saves changes locally with a message.
6. **git config --global user.name "Sneha Tumaskar"** → Sets global Git username for commits.
7. **git config --global user.email "snehatumaskar@gmail.com"** → Sets global Git email for commits.
8. **git log** → Displays commit history of the repository.
9. **git config --global user.name** → Displays the configured Git username.
10. **git config --global user.email** → Displays the configured Git email.
11. **git branch -M main** → Renames the current branch to "main".

Github Connection Commands :

12. **git remote add origin URL** → Connects local repository to a GitHub repository.
13. **git push -u origin main** → Uploads code to GitHub and sets upstream branch.
14. **git branch feature-ui** → Creates a new branch named "feature-ui".
15. **git branch** → Lists all branches in the repository.
16. **git pull origin main** → Fetches and merges latest changes from GitHub.

1. Flutter Architecture

Everything is a Widget

In Flutter, everything is a widget.
UI elements like buttons, text, images, layouts, and even the app itself are widgets.

Example:

- Text → Widget
- Button → Widget
- Screen → Widget
- App → Widget

This makes Flutter highly flexible and reusable.

Widget Tree Concept

Flutter UI is built using a tree structure of widgets.

Example:

```
MaterialApp
 └── Scaffold
     ├── AppBar
     └── Body
         └── Column
```

```
    ├── Text
    └── Button
```

Each widget is connected like a tree node.

Build Method

```dart
Widget build(BuildContext context) {
  return Text("Hello Flutter");
}
```

Key Points:

- It describes UI structure
- Called automatically when UI updates
- Returns widgets

Hot Reload Mechanism

Hot Reload allows instant UI updates without restarting the app.

Benefits:

- Fast development
- Keeps app state
- Saves time

2. Widget Basics

StatelessWidget

A widget that does not change state.

Example:

```dart
class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text("Static UI");
  }
}
```

Features:

- Immutable
- Fast
- Used for static UI

StatefulWidget

A widget that can change state during runtime.

Example:

```
class Counter extends StatefulWidget {
  @override
  _CounterState createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Text("Count: $count");
  }
}
```

Features:

- Dynamic UI
- Uses setState()
- Stores data

BuildContext

BuildContext tells Flutter where a widget is located in the widget tree.

Uses:

- Navigation
- Theme access
- Layout positioning

Key Concept

Keys help Flutter identify widgets uniquely.

Benefits:

- Prevents UI confusion
- Improves performance
- Maintains state correctly

3. Basic Widgets

MaterialApp

Root widget of Flutter app.

```
MaterialApp(
  home: MyHomePage(),
)
```

Scaffold

Provides structure to screen.

Includes:

- AppBar
- Body
- Floating button

AppBar

Top bar of the app.

Text / RichText

Used to display text.

RichText allows styled text.

Container

Used for styling and layout.

Features:

- Padding
- Margin
- Color
- Border

Row / Column

Row

Horizontal layout

Column

Vertical layout

Stack

Used to place widgets on top of each other.

Example:

- Image + Text overlay

Padding / Center

- Padding → spacing
- Center → align in middle

4. Layout Widgets

SizedBox

Used for fixed spacing.

Expanded / Flexible

- Expanded → takes full available space
- Flexible → adjusts size dynamically

Spacer

Creates flexible empty space between widgets.

ListView

Scrollable list of items.

Example uses:

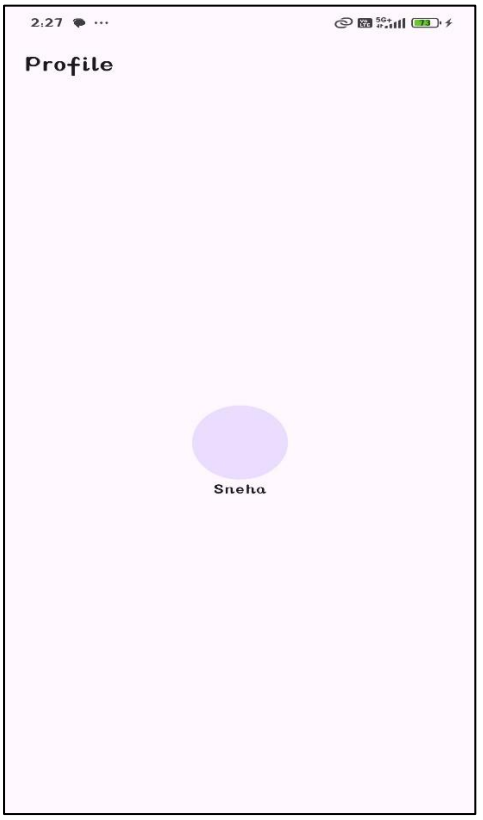
- Product list
- Messages
- Contacts

1. Profile Card :

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    home: ProfileScreen(),
  )); // MaterialApp
}

class ProfileScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Profile")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CircleAvatar(radius: 40),
            Text("Sneha"),
          ],
        ), // Column
      ), // Center
    ); // Scaffold
  }
}
```

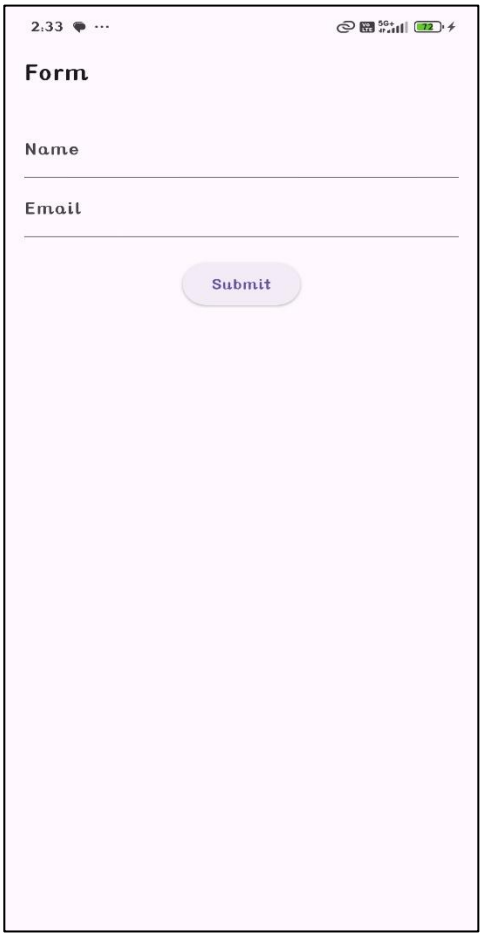


2. Form :

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    home: FormScreen(),
  )); // MaterialApp
}

class FormScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Form")),
      body: Padding(
        padding: EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              decoration: InputDecoration(labelText: "Name"),
            ), // TextField
            TextField(
              decoration: InputDecoration(labelText: "Email"),
            ), // TextField
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {},
              child: Text("Submit"),
            ) // ElevatedButton
          ],
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```



3. List

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    home: ListScreen(),
  )); // MaterialApp
}

class ListScreen extends StatelessWidget {
  final items = ["Apple", "Banana", "Mango", "Orange", "Grapes"];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("List")),
      body: ListView(
        children: items.map((e) => ListTile(title: Text(e))).toList(),
      ), // ListView
    ); // Scaffold
  }
}
```